Academic Research Methodology (31.10.2026)
Department of Information Technology —
Instructor: Dalya Raad AlAni

LAB MANUAL 6: TYPESETTING ALGORITHMIC LOGIC, SUB-NETWORK LEDGERS, & RUNTIME TENSE VERIFICATION

Lab Six

1 Part A: LaTeX Typesetting Analytical & Algorithmic Logic

In computer science research and graduation theses, technical data representation—such as IP address allocation grids, network packet performance indices, and database entity-relationship models—constitute primary empirical findings. Students must deploy these tabular structures natively within the canvas layout without causing cell overflow, margin violation, or layout fragmentation.

1.1 Network Architecture Data Representation

When formatting networking analytics (e.g., IPv4 subnet allocations or routing data), explicit layout boundaries prevent textual overlapping. The `tabularx` package framework layer enables the definition of a total bounding width of `\textwidth` while deploying flexible column parameters to handle text wrapping dynamically.

1.2 Database Entity-Relationship Modeling Fields

To render database schemata and structural relational attributes cleanly without cluttering the page layout geometry, models must employ the rules defined by the `booktabs` engine. This setup enforces formal data visualization standards using professional spacing parameters (`\toprule`, `\midrule`, and `\bottomrule`) instead of raw vertical dividing lines.

2 Part B: The Runtime Behavior Tense Workshop

Documenting operational system setups requires absolute technical clarity. When describing the performance of algorithms or network configurations under active execution conditions, text blocks must function within consistent grammatical limits.

2.1 Enforcing Factual Present Tense Layouts

Unlike the historical description of closed baseline experiments, describing active system runtime traits under specified conditional parameters demands the use of the Present Simple tense:

- **The Runtime Truth Principle**: A compiled system architecture operates as an enduring technical fact. Verbs describing code execution (e.g., *updates, routes, authenticates*) must reflect an active state of operational reality.
- **Structuring Conditional Parameters**: Conditional expressions (e.g., *if/when* strings combined with present-tense indicators) are required to map structural software forks cleanly.

Copyright © 2026 Dalya Raad AlAni

- **Eliminating Outdated Past Assertions**: Writing that a loop *cleared* a buffer implies the function executed once during an isolated test and is no longer valid.

3 Part C: Core Practical Case Studies

3.1 Case Study (a): Algorithmic Logic and Structured Allocation Mapping

Students will compile the following baseline code to verify that the table structures, array widths, algorithmic expressions, and pseudo-code frames handshake properly without overflowing the layout margins.

1. Launch TEXstudio and open a completely new empty editor canvas.
2. Copy, paste, and analyze the structured template script shown below:

```
\documentclass[11pt,a4paper]{article}
\usepackage[top=0.8in, bottom=0.8in, left=1.0in, right
   =1.0in]{geometry}
\usepackage{booktabs}
\usepackage{tabularx}
\usepackage{listings}
\usepackage{xcolor}
\usepackage{amsmath}

\begin{document}

        \section{Algorithmic Logic and Allocation
           Mapping}
        The proposed network engine isolates routing
           processes from tracking sub-tiers using a
           secure configuration layout.

        \subsection{Network Addressing Ledger}
        The physical allocation layer maps the
           addressing parameters across environments as
            illustrated in Table~\ref{tab:net_matrix}.

        \begin{table}[h]
                \centering
                \caption{Subnet Infrastructure
                   Configuration and Compliance Mapping
                   .}
                \label{tab:net_matrix}
                \vspace{0.2cm}
                \noindent\begin{tabularx}{\textwidth}{
                   ll c >{\raggedright\arraybackslash}X
                   }
                        \toprule
                        \textbf{Subnet Domain} & \
                           textbf{IP Address Range} & \
                           textbf{VLAN} & \textbf{GDPR
                           Compliance Action} \\
                        \midrule
                        Edge\_Tier\_01 & \texttt
                           {192.168.10.0/24} & 100 &
                           Local node logging with
                           masked client octets. \\
```

```
                        Auth\_Gateway  & \texttt
                            {10.0.5.0/28}     & 105 &
                            Cryptographically salted
                            token tracking layer. \\
                        Database\_Core & \texttt
                            {172.16.32.0/22}  & 200 &
                            Complete hashing protection
                            for database records at rest
                            . \\
                        \bottomrule
                \end{tabularx}
        \end{table}

        \subsection{Relational Entity-Relationship
            Attribute Bounds}
        The data structures are configured using
            optimized typing constraints. Table~\ref{tab
            :db_matrix} outlines the operational schema
            parameters, where validation loops verify
            authentication credentials with a worst-case
             validation complexity defined by asymptotic
             algorithmic processing strings as $\mathcal
            {O}(n \log n)$.

        \begin{table}[h]
                \centering
                \caption{Database Schema Matrix Fields
                    and Security Rules.}
                \label{tab:db_matrix}
                \vspace{0.2cm}
                \noindent\begin{tabularx}{\textwidth}{l
                     p{2.8cm} l >{\raggedright\
                    arraybackslash}X}
                        \toprule
                        \textbf{Entity Field Key} & \
                            textbf{SQL Data Type} & \
                            textbf{Constraint} & \textbf
                            {ISO/IEC 27001 Cryptographic
                             Rule} \\
                        \midrule
                        \texttt{user\_id}      & \
                            texttt{VARCHAR(64)}   &
                            PRIMARY KEY & Enforces role-
                            based unique tracking
                            attributes. \\
                        \texttt{pass\_hash}    & \
                            texttt{CHAR(128)}     & NOT
                            NULL    & Stores SHA-512
                            hashes generated at user
                            registration. \\
                        \texttt{session\_tok}  & \
                            texttt{VARBINARY(32)} &
                            UNIQUE      & Cryptographic
                            validation tokens rotated
                            every 15 minutes. \\
                        \bottomrule
                \end{tabularx}
        \end{table}
```

```
                        \subsection{Token Validation Runtime Routine}
                        The code routine shown in Listing~\ref{lst:
                           token_loop} details the verification loops
                           executing within the authentication
                           perimeter layer.

                        \begin\{lstlisting\}[language=Python, basicstyle
                           =\textsl{\textbackslash small\textbackslash
                           ttfamily}, breaklines=true, backgroundcolor
                           =\textsl{\textbackslash color\{gray!10\}},
                           frame=single, caption=\{Runtime token
                           security loop verification routine.\}, label
                           =\{lst:token_loop\}]
                        def verify_access_token(client_request):
                        # Check conditional authentication parameter
                           criteria
                        if not client_request.has_valid_token():
                        log_security_event("Unauthorized Access Attempt
                            Detected")
                        return "Access Denied: Invalid Security Token"

                        if client_request.is_expired():
                        return "Access Denied: Token Session Expired"

                        return "Access Granted: Routing to Database
                           Tier"
                        \end\{lstlisting\}

                        \end\{document\}
```

3. Save the source file as `lab6_baseline.tex` inside your course repository folder.

4. Press the **F5 key** on your keyboard to trigger the full Build & View pipeline execution.

Copyright © 2026